

Lab 3

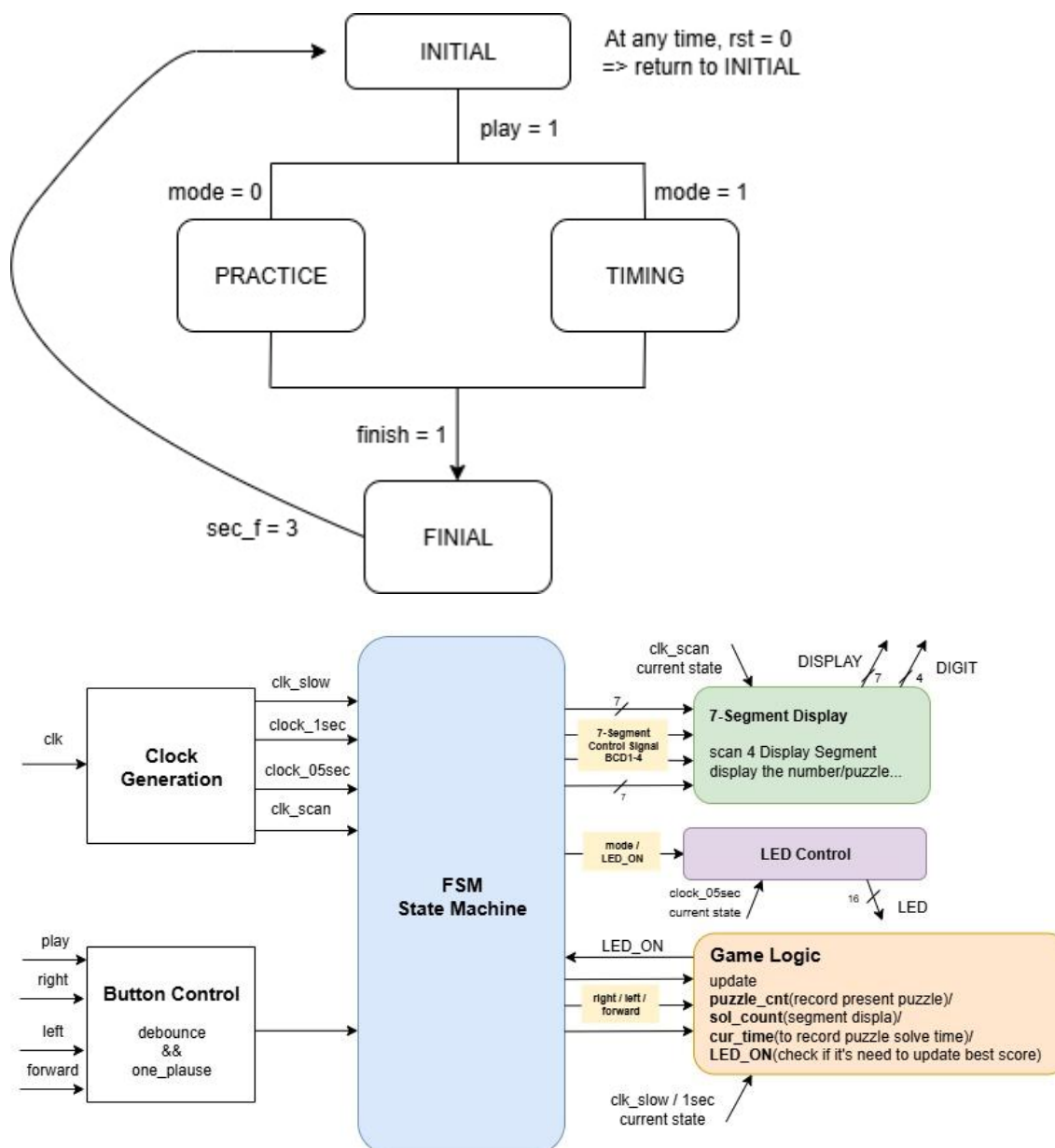
學號: 111062118

姓名: 江佩霖

A. Lab Implementation

1. Lab3_Adv1

(1) Block diagram/FSM state diagram



(2) 程式碼架構、實現邏輯

```
// ===== //
// LED Flashing Logic //
// ===== //
// LED ON to control the light should hold or flash
always @(*) begin
    if(cs == FINAL && solved)begin
        if(mode == 1'd0)begin
            if(flash)LED = 16'hFFFF;
            else LED = 16'h0;
        end
        else begin
            if(LED_ON)LED = 16'hFFFF;
            else begin
                if(flash)LED = 16'hFFFF;
                else LED = 16'h0;
            end
        end
    end
    else begin
        LED = 16'h0;
    end
end
```

LED_ON 會由遊戲邏輯 block 更新，後由 LED 顯示 block 會依照此訊號決定 LED 要閃爍還是持續保持亮燈。

```
// 根據 puzzle_cnt 和解題狀態決定顯示分數
always @(*) begin
    case(puzzle_cnt)
        2'd0:begin
            if(p1_solved)begin
                case(score1_h) ...
            endcase
            case(score1_l) ...
            endcase
        end
        2'd1:begin
            if(p2_solved)begin
                case(score2_h) ...
            endcase
            case(score2_l) ...
            endcase
        end
        2'd2:begin
            if(p3_solved)begin
                case(score3_h) ...
            endcase
            case(score3_l) ...
            endcase
        end
        default:begin ...
    endcase
end
```

遊戲邏輯 block 中，會透過 puzzle_cnt(當前 puzzle)以及記憶各個 puzzle 的 solved 訊號(是否解開此 puzzle)來顯示分數。

```

case(puzzle_cnt)
2'd0:begin
  case(sol_count)
4'd0:begin
    BCD1 <= 7'b0100001;
    BCD2 <= puzzle1_2;
  end
4'd1:begin
    if(sec_count == 1'd1)begin
      BCD1 <= 7'b0110001;
      BCD2 <= puzzle1_2;
    end
  end
4'd2:begin
    if(sec_count == 1'd1)begin
      BCD1 <= 7'b0111001;
      BCD2 <= puzzle1_2;
    end
  end
4'd3:begin
    if(sec_count == 1'd1)begin
      BCD1 <= 7'b0111101;
      BCD2 <= puzzle1_2;
    end
  end
4'd4:begin
    if(sec_count == 1'd1)begin
      BCD1 <= 7'b0111111;
      BCD2 <= puzzle1_2;
    end
  end
  //sol_count = 5 means solved, should go to final state
4'd5:begin
    if(sec_count == 1'd1)begin
      BCD1 <= 7'b1111111;
      BCD2 <= puzzle1_2;
    end
  end
end
endcase

```

透過 sol_count 來決定當前的 puzzle 解到哪一步，並且顯示相應的圖樣。

Sol_count 則由另一個 block 控制是否輸入正確的訊號(forward, right)，判斷是否更新此 sol_count 數值。

```

// Update the record score & LED_ON logic
always @(posedge clk_slow or posedge rst) begin
  if(rst)begin...
  end
  else begin
    if(cs == INITIAL)begin...
    end
    else begin
      if(finish && mode == 1'd1 && entered == 1'd0)begin
        entered <= 1'd1;
        case(puzzle_cnt)
        2'd0:begin
          if(score1 >= cur_time)begin
            score1 <= cur_time;
            LED_ON <= 1'd0; // let LED flash
          end
          else begin
            LED_ON <= 1'd1; // keep LED light
          end
        end
        2'd1:begin...
        end
        2'd2:begin...
        end
      endcase
    end
  end
end
end
end

```

當 puzzle 完成時，會發出 finish 的訊號，遊戲邏輯 block 會根據 mode 以及當前的 cur_time 以及紀錄的 score 來比較是否要更新數值。若為更高的紀錄則會設置 LED_ON，使 LED display block 正確運作。

B. Questions and Discussions

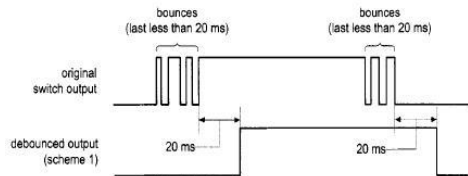
1. Why are debounce and one-pulse modules required? How does the FSM's clock rate relate to the one-pulse module's clock rate? Explain your reason. What issues occur if the wrong clock rate is used? (Illustrate with waveforms to explain.)

Ans.

(1) -1 Debounce module:

由於實體按鈕在按下或是釋放時，金屬的接觸點會產生彈跳的現象，這會導致輸入的信號在 High 和 Low 之間快速不穩定的跳動。若 FSM 或是計數器直接使用這個彈跳的信號，會將這些快速的跳動判定為多次的按壓，導致錯誤的操作或輸出。

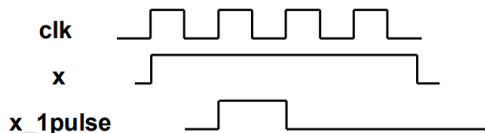
Debounce module 透過延遲來過濾這些短暫的彈跳訊號，確保輸入訊號穩定的保持在 High 或 Low 超過一訂時間後才輸出。



如圖，當按鈕按下以及被釋放時，均會有一小段訊號不穩定的狀況，debounce module 會等待其訊號穩定後將穩定的訊號輸出，避免不穩定的訊號被 FSM 或計數器接收，導致不正確的行為。

-2 One-Pulse module:

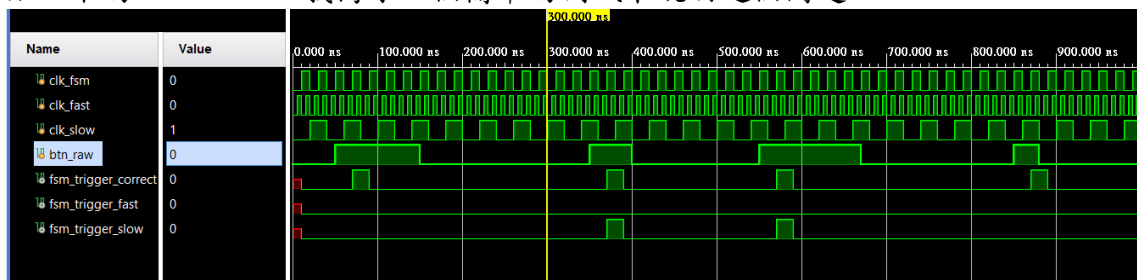
經過 Debounce module 的訊號，在按鈕按下的整個持續時間中都會是一個持續的 High 信號，但在許多應用中只需要單次事件觸發，而不是按鈕被按下的期間持續的觸發。若 FSM 或計數器錯誤的使用這個訊號，可能會在按鈕按住的期間每個 clk cycle 都重複觸發事件，導致多次不必要的狀態轉換或計數。



如圖，當 x 訊號被拉起(High)，若沒有 one_pulse module，訊號很可能在每個 positive edge clk 或是 negative edge 被 FSM 或計數器接收；而 one_pulse 會將訊號轉變成一個 cycle 的穩定訊號，避免不正確的行為發生。

- (2) 讓 One-Pulse 用與 FSM 相同的時脈 (same clock domain) 執行。這樣生成的脈衝會在 FSM 的時脈邊緣出現，FSM 不會遺漏或多次觸發。

- (3) 如以下的 waveform，我寫了一個簡單的測試來說明這個問題。



當 One-Pulse 以較快時脈產生時，脈衝寬度過短，FSM 時脈取樣時常錯過高電位，導致完全未觸發。當 One-Pulse 以較慢時脈產生時，雖然脈衝寬度較長，但由於時脈不同步，按鍵變化可能發生在慢時脈未取樣的區間，因此出現漏觸發的情況。

2. How do you control and record the pen's position in the puzzle? Explain clearly and provide partial code examples.

Ans.

在我的設計中，我直接將謎題的筆路徑寫到 code 中，用 sol_count 這個變數紀錄當前 puzzle 解謎的狀況(走到第幾步)，系統會根據這個值來決定顯示器該顯示什麼，並相應的更新筆的位置。

Partial example of the code structure:

```
// Record the current puzzle progress
case (puzzle_cnt)
  2'd0: begin
    case (sol_count)
      4'd0: begin
        if (forward)
          sol_count <= sol_count + 4'd1;
        end
      4'd1: begin
        if (right)
          sol_count <= sol_count + 4'd1;
        end
      4'd2: begin
        if (right)
          sol_count <= sol_count + 4'd1;
        end
      4'd3: begin
        if (forward)
          sol_count <= sol_count + 4'd1;
        end
      4'd4: begin
        if (right) begin
          finish <= 1'd1;
          sol_count <= sol_count + 4'd1;
        end
      end
    default: begin
      sol_count <= sol_count;
      finish <= 1'd0;
    end
  endcase
end
endcase
```

puzzle_cnt 會紀錄是哪一個 puzzle，而 sol_count 則是記錄當前解謎的狀態。

```
case(puzzle_cnt)
  2'd0:begin
    case(sol_count)
      4'd0:begin
        BCD1 <= 7'b0100001;
        BCD2 <= puzzle1_2;
      end
      4'd1:begin
        if(sec_count == 1'd1)begin
          BCD1 <= 7'b0110001;
          BCD2 <= puzzle1_2;
        end
      end
      4'd2:begin
        if(sec_count == 1'd1)begin
          BCD1 <= 7'b0111001;
          BCD2 <= puzzle1_2;
        end
      end
      4'd3:begin
        if(sec_count == 1'd1)begin
          BCD1 <= 7'b0111101;
          BCD2 <= puzzle1_2;
        end
      end
      4'd4:begin
        if(sec_count == 1'd1)begin
          BCD1 <= 7'b0111111;
          BCD2 <= puzzle1_2;
        end
      end
      //sol_count = 5 means solved, should go to final state
      4'd5:begin
        if(sec_count == 1'd1)begin
          BCD1 <= 7'b1111111;
          BCD2 <= puzzle1_2;
        end
      end
    endcase
  end
end
```

這邊是顯示的部分，會根據 sol_count 來直接指定顯示器的顯示狀況。

C. Problem Encountered

1. 設計複雜

由於顯示器需要較慢的 clk 來更新數值，其他地方的設計亦需要各種不同的 clk 做驅動 (比如說要有計時器以及 0.5 秒鐘的閃爍 LED 等等)，導致各種不同的時序很難設 debug。後來統一使用較慢的時序 clk_slow，並且加上其他判斷的邏輯來管理所有的顯示器、LED，確保在有正確顯示的狀況下能正確更新數值。

```
// ===== //
// Game Logic and Timing //
// ===== //
// Puzzle Selection Logic - INITIAL
// Use puzzle_cnt to track current puzzle (0, 1, or 2), and display
> always@(posedge clk_slow or posedge rst)begin...
end
// control solved signal
> always @(posedge clk_slow or posedge rst) begin...
end
```

2. 較複雜的遊戲設計，導致 debug 的難度增加

將每個不同的 block 加上詳細的註解，並且分別記錄在不同的區塊內，這樣能更快找到對應的區塊更動，debug 上也更容易。

```
// ===== //
// LED Flashing Logic //
// ===== //
// LED ON to control the light should hold or flash You, 9 hours ago •
> always @(*) begin...
end

// ===== //
// 7-Segment Display //
// ===== //
// Scan 4 digits to display
> always@(posedge clk_scan or posedge rst)begin...
end
// Display encoding for current time
> always@(*)begin...
end
> always@(*)begin...
end
// 根據 puzzle_cnt 和解題狀態決定顯示分數
> always @(*) begin...
end
// Update 7-segment display values based on state and puzzle
> always @(posedge clk_slow or posedge rst) begin...
end

// ===== //
// State Machine Control //
// ===== //
// clk trigger for state transition
> always @(posedge clk_slow or posedge rst)begin...
end
> always@(*)begin...
end
```

D. Suggestions

我覺得很棒，順便跟助教們說辛苦了...。

皮卡丘站起來變成什麼？

皮卡兵